ASSESSMENT TOOL 2 – SOLUTIONS

NAME : RAGAVI S
REGISTER NO : 192565027
COURSE CODE : CSA1717
COURSE NAME : ARTIFICIAL INTELLIGENCE

QUESTION 1: Constraint Satisfaction Problem (Doctor Shift Scheduling)

1. Problem Modeling & Theoretical Framework

A Constraint Satisfaction Problem (CSP) provides a formal mathematical framework for modeling assignment problems with discrete variables restricted by unary and binary constraints. In this problem, we are tasked with scheduling three doctors into three temporal shifts.

- Variables (X): X = {D1, D2, D3}, where each variable represents a specific doctor.
- Domains (D): D = {Morning (M), Afternoon (A), Night (N)} for all variables.
- Constraint 1 (Unary): D1 cannot work Night shift (D1 != Night).
- Constraint 2 (Binary Precedence): D2 must work before D3 in shift ordering (D2 < D3, where Morning < Afternoon < Night).
- Constraint 3 (All-Different / Capacity): Only one doctor per shift (D1 != D2, D1 != D3, D2 != D3).
- Constraint 4 (Unary): D3 cannot work Morning shift (D3 != Morning).
- Constraint 5 (Completeness): Every doctor must be assigned exactly one shift.

2. Mathematical Formulation & Domain Reduction (Node Consistency)

Before initiating tree-based backtracking search, we apply Node Consistency (Unary Constraint Filtering) to prune invalid domain values from the initial state space:

- Domain(D1) = {Morning, Afternoon, Night} - {Night} = {Morning, Afternoon}
- Domain(D2) = {Morning, Afternoon, Night}
- Domain(D3) = {Morning, Afternoon, Night} - {Morning} = {Afternoon, Night}

This initial reduction shrinks the unconstrained state space from 3^3 = 27 possibilities down to 2 x 3 x 2 = 12 search states.

3. Step-by-Step Backtracking Tree Execution

Backtracking search explores the state space depth-first, maintaining a partial assignment and validating all constraints at each step.

Step 3.1: Variable Assignment for Doctor 1 (D1)

- Select unassigned variable D1 from reduced domain {Morning, Afternoon}.
- Option 1.1: Assign D1 = Morning.
  - Constraint Verification: Satisfies Constraint 1 (D1 != Night). Partial assignment {D1 = Morning} is valid.

Step 3.2: Variable Assignment for Doctor 2 (D2)

- Current Partial Assignment: {D1 = Morning}. Select variable D2 from domain {Morning, Afternoon, Night}.
- Option 2.1: Try D2 = Morning.
  - Check Constraint 3 (All-Different): D1 = Morning and D2 = Morning -> VIOLATION (Shift already occupied). Prune branch.
- Option 2.2: Try D2 = Afternoon.
  - Check Constraint 3 (All-Different): D1 = Morning, D2 = Afternoon -> VALID.
  - Check Constraint 2 Precedence (D2 < D3): Since D2 = Afternoon, D3 must be assigned a shift strictly after Afternoon (i.e., Night). VALID so far.
  - Accept partial assignment: {D1 = Morning, D2 = Afternoon}.

Step 3.3: Variable Assignment for Doctor 3 (D3)

- Current Partial Assignment: {D1 = Morning, D2 = Afternoon}. Select variable D3 from domain {Afternoon, Night}.
- Option 3.1: Try D3 = Afternoon.
  - Check Constraint 3 (All-Different): D2 = Afternoon and D3 = Afternoon -> VIOLATION. Prune branch.
- Option 3.2: Try D3 = Night.
  - Check Constraint 4 (Unary): D3 != Morning -> VALID (D3 = Night).
  - Check Constraint 2 Precedence (D2 < D3): D2 (Afternoon) < D3 (Night) -> VALID.
  - Check Constraint 3 (All-Different): D1=M, D2=A, D3=N are all distinct -> VALID.
  - Check Constraint 5 (Completeness): All 3 variables assigned -> SOLUTION FOUND.

4. Final Valid Schedule & Constraint Validation Summary

The backtracking algorithm terminates with a unique, complete, and fully consistent schedule:

Doctor Variable	Assigned Shift	Constraint Satisfaction Proof
Doctor 1 (D1)	Morning Shift	Satisfies D1 != Night and

		shift uniqueness (D1 != D2, D1 != D3)
Doctor 2 (D2)	Afternoon Shift	Satisfies precedence D2 < D3 (Afternoon < Night) and D1 != D2
Doctor 3 (D3)	Night Shift	Satisfies D3 != Morning, D2 < D3, and shift uniqueness

QUESTION 2: Grid Navigation using Best-First / A* Search

1. Problem Environment & Heuristic Modeling

The navigation problem takes place in a 2D discrete grid of dimensions 5 x 5 (25 possible grid states), indexed using standard Cartesian coordinates (x, y) where x, y in {0, 1, 2, 3, 4}.

- Start State (S): Coordinate (0, 0)
- Goal State (G): Coordinate (4, 4)
- Obstacle Barrier: Occupies grid positions (1, 1), (2, 1), and (3, 1), forming a vertical wall.
- Actions & Cost: Discrete transitions Up (-1, 0), Down (+1, 0), Left (0, -1), Right (0, +1). Each valid transition has unit cost g(n) = 1.
- Heuristic Function h(n): Manhattan Distance to Goal (4, 4), defined as h(n) = |x_n - 4| + |y_n - 4|.
- Evaluation Function f(n): f(n) = g(n) + h(n), representing total estimated path cost passing through node n.

2. Detailed Step-by-Step Node Expansion & Priority Queue Trace

Step 2.1: Initialization

- Compute initial metrics for Start Node (0, 0): g(0,0) = 0, h(0,0) = |0-4| + |0-4| = 8, f(0,0) = 8.
- Initialize OPEN Priority Queue: [(0,0) | g=0, h=8, f=8].
- Initialize CLOSED Set: {}.

Step 2.2: Iteration 1

- Pop node with lowest f(n): (0, 0). Move (0, 0) to CLOSED.
- Generate valid adjacent neighbors:
  - Down (1, 0): g = 0+1 = 1, h = |1-4|+|0-4| = 7 -> f(1,0) = 1+7 = 8.
  - Right (0, 1): g = 0+1 = 1, h = |0-4|+|1-4| = 7 -> f(0,1) = 1+7 = 8.
- OPEN Priority Queue: [(1,0) f=8, (0,1) f=8].

Step 2.3: Iteration 2

- Pop node (1, 0) from OPEN. Move (1, 0) to CLOSED.
- Generate valid adjacent neighbors:
  - Down (2, 0): g = 1+1 = 2, h = |2-4|+|0-4| = 6 -> f(2,0) = 2+6 = 8.
  - Right (1, 1): OBSTACLE DETECTED. Blocked path, discard neighbor.
  - Up (0, 0): Already in CLOSED, ignore.
- OPEN Priority Queue: [(2,0) f=8, (0,1) f=8].

Step 2.4: Iteration 3

- Pop node (2, 0) from OPEN. Move (2, 0) to CLOSED.
- Generate valid adjacent neighbors:
  - Down (3, 0): g = 2+1 = 3, h = |3-4|+|0-4| = 5 -> f(3,0) = 3+5 = 8.
  - Right (2, 1): OBSTACLE DETECTED. Blocked path, discard neighbor.
- OPEN Priority Queue: [(3,0) f=8, (0,1) f=8].

Step 2.5: Iteration 4

- Pop node (3, 0) from OPEN. Move (3, 0) to CLOSED.
- Generate valid adjacent neighbors:
  - Down (4, 0): g = 3+1 = 4, h = |4-4|+|0-4| = 4 -> f(4,0) = 4+4 = 8.
  - Right (3, 1): OBSTACLE DETECTED. Blocked path, discard neighbor.
- OPEN Priority Queue: [(4,0) f=8, (0,1) f=8].

Step 2.6: Iteration 5 (Rounding the Obstacle)

- Pop node (4, 0) from OPEN. Move (4, 0) to CLOSED.
- Generate valid adjacent neighbors:
  - Right (4, 1): Clear path! g = 4+1 = 5, h = |4-4|+|1-4| = 3 -> f(4,1) = 5+3 = 8.
- OPEN Priority Queue: [(4,1) f=8, (0,1) f=8].

Step 2.7 to Step 2.9: Moving Towards Goal

- Iteration 6: Expand (4, 1) -> Generates (4, 2) [g=6, h=2, f=8].
- Iteration 7: Expand (4, 2) -> Generates (4, 3) [g=7, h=1, f=8].
- Iteration 8: Expand (4, 3) -> Generates (4, 4) [g=8, h=0, f=8].
- Iteration 9: Pop (4, 4) from OPEN. Goal condition met!

3. Search Output & Path Analysis

The A* search algorithm successfully bypasses the obstacle wall and constructs the optimal route with optimal cost:

- Optimal Path Sequence: (0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)

- Total Path Length: 8 transitions
- Total Path Cost: g(Goal) = 8 units

QUESTION 3: Autonomous Rescue Robot in Partially Observable Environment

1. Constraint Analysis & Decision-Making Impact

Operating an autonomous system in disaster environments requires adapting standard AI search paradigms to handle limited sensing, uncertain obstacles, and non-uniform traversal costs.

- Four-Directional Movement (Up, Down, Left, Right): Restricts movement to discrete grid steps, requiring the agent to navigate around orthogonal barriers rather than diagonal shortcuts.
- Blocked Cells (Obstacles): Random room collapses physically block paths. Because obstacle locations are unknown prior to arrival, the agent must detect dead ends locally and re-route dynamically.
- Limited Sensing Capability: The agent can only perceive immediate adjacent cells. This partial observability invalidates offline search algorithms (like standard offline A*), requiring an Online Search Agent that interleaves perception, planning, and action execution.
- Variable Traversal Costs (Standard = 1, Risky Zone = 3): Standard BFS assumes equal edge weights (cost = 1) and minimizes step count, which would force the agent into high-risk disaster zones. Uniform Cost Search (UCS) expands nodes by cumulative path cost g(n), ensuring the robot prioritizes safe routes over shorter, dangerous ones.
- Dynamic Knowledge Base Updates: The robot maintains a local topological map of visited cells and step costs to avoid looping infinitely in unknown spaces.

2. AI Concepts Application & PEAS Framework

PEAS Architecture Profile

- Performance Measure: Maximize survivor rescue safety, minimize total cumulative path cost g(n), avoid risky disaster cells, and reach Goal state without agent destruction.
- Environment: Disaster building represented as a 2D discrete grid, partially observable, dynamic obstacles, non-uniform step costs.
- Actuators: Directional drive wheel motors (Move Up, Down, Left, Right).
- Sensors: Proximity range sensors, infrared/heat sensors for localized survivor detection, and terrain cost evaluation sensors.

Agent Rationality

The robot behaves rationally by making decisions that maximize its expected performance measure given its current percept history and local dynamic map. Rather than recklessly taking shortest paths through dangerous structural collapses, a rational agent balances safety against step distance using cost-weighted search.

Environment Classification Matrix

Environment Property	Classification & Technical Justification
Observability	Partially Observable - Sensors can only perceive adjacent cells; global map is hidden.
Determinism	Deterministic - Action execution outcome is reliable (moving right lands on right cell).
Episodic vs. Sequential	Sequential - Current navigation choices directly influence future positioning and path options.
Static vs. Dynamic	Dynamic / Dynamic Knowledge - Terrain condition and obstacles must be sensed dynamically during execution.

3. Online Uniform Cost Search (UCS) Agent Execution Trace

Let the environment be represented as a 4x4 room grid from (0,0) to (3,3). Start state S = (0,0), Goal G = (3,3). Obstacles block (1,0) and (1,1). Risky zone (cost = 3) is located at (0,2).

Simulated Execution Output of Online UCS Agent

[Current Position]: (0, 0) | Cumulative Cost: 0
Percept: Scanning adjacent cells...
Down (1, 0): OBSTACLE DETECTED
Right (0, 1): Clear, Cost = 1
-> Action: Execute Move Right to (0, 1)

[Current Position]: (0, 1) | Cumulative Cost: 1
Percept: Scanning adjacent cells...
Down (1, 1): OBSTACLE DETECTED
Right (0, 2): RISKY ZONE DETECTED (Cost = 3, Total g = 4)
Left (0, 0): Already Visited

```
  Down-Right Alternative Route evaluation...
  -> Action: Execute Move Right to (0, 2) [Cost = 3]

[Current Position]: (0, 2) | Cumulative Cost: 4
  Percept: Scanning adjacent cells...
  Down (1, 2): Clear, Cost = 1
  -> Action: Execute Move Down to (1, 2)

[Current Position]: (1, 2) | Cumulative Cost: 5
  -> Moving Down to (2, 2) [Cost = 1, Total g = 6]
  -> Moving Down to (3, 2) [Cost = 1, Total g = 7]
  -> Moving Right to (3, 3) [Cost = 1, Total g = 8]

--- RESCUE OPERATION COMPLETE ---
Path Traversed: (0,0) -> (0,1) -> (0,2) -> (1,2) -> (2,2) -> (3,2) -> (3,3)
Total Path Cost: 8
```

4. Solution Justification & Comparative Analysis

- Why Online Search? Offline search algorithms require complete global map information before computing a plan. In a disaster building with partial observability, an offline agent would create invalid plans that immediately fail upon hitting an unmapped obstacle. Online search interleaves sensing, planning, and moving.
- Why Uniform Cost Search (UCS) over BFS? Breadth-First Search (BFS) is optimal only when all step costs are equal. When entering risky zones costs 3 vs standard cost 1, BFS would pick paths with fewer steps regardless of total risk. UCS guarantees finding the path with the minimum total cost g(n).